Intelligent Public Transport and Crowd Prediction System: Real-Time Bus Tracking with Machine Learning-Based Crowd Level Prediction

Vaishnav Bhairu Tevaratti[1], Umay Parthsarthi Sawairam[2], Aryan Dattatray Waifalkar[3], Sandesh Vijay Suryawanshi[4], Shubham Suresh Powar[5], Dr. P. M. Gavali[6]

[12345]Undergraduate Student, [6]Assistant Professor
[123456]Department of Computer Science and Engineering,
[123456]D.K.T.E. Society's Textile and Engineering Institute (An Autonomous Institute), Ichalkaranji, India
[1]vaishnavtevaratti@gmail.com, [2]umaysawairam07@gmail.com, [3]aryanwaifalkar7049@gmail.com,
[4]sandeshsuryawanshi415@gmail.com, [5]shubhampowar602@gmail.com, [6]gavalipm87@gmail.com

Abstract - City bus users usually travel without two pieces of information that matter most to them: the present position of the vehicle they are waiting for and the likelihood that it will have room for them. This work describes the design, implementation, and evaluation of an Intelligent Public Transport and Crowd Prediction System that supplies both through an ordinary web browser. The platform draws each bus on a live map, refreshes its position every five seconds through a persistent WebSocket channel instead of repeated polling, and labels every trip with an expected crowding category of Low, Medium, or High. The category is produced by a separate Python inference service that learns from calendar and route attributes, namely hour, weekday, weekend flag, route length, number of stops, and vehicle capacity. Administrators maintain the fleet and the route network through the same application under role-based access control. The software is organised in five layers: a React client, a Spring Boot application server, a MySQL store, the learning service, and the push channel. Three classifiers were compared under a chronological train-test split of 60,000 simulated trips; histogram-based gradient boosting gave the best cross-validated macro F1-score and reached 92.1% accuracy (macro F1 0.888) on unseen days, with every error falling between adjacent categories. A companion model that forecasts occupancy fifteen minutes ahead lowered the mean absolute error from 2.93 to 2.12 passengers relative to a no-change baseline. Because the training records are synthetic, these figures validate the pipeline rather than field accuracy, and the modular design is intended to accept real positioning feeds and passenger counts in future deployments.

Index Terms - Intelligent transportation systems, bus crowding, occupancy forecasting, gradient boosting, real-time passenger information, WebSocket, Spring Boot, React.

I. INTRODUCTION

Buses carry a large share of daily trips in Indian cities because they are inexpensive, reach most neighbourhoods, and require no investment from the rider. Their attractiveness, however, depends less on fares than on predictability. A commuter who cannot tell whether the next vehicle is two minutes or twenty minutes away, or whether it will arrive with standing room only, tends to leave early, wait anxiously, or shift to a private vehicle. Each of these responses erodes the social and environmental value that a shared transport network is supposed to deliver.

Two kinds of uncertainty can be separated. The first is spatial: where the vehicle is right now and when it will reach the stop. The second is volumetric: how many people are already on board and how many will board before the rider does. Earlier passenger information work concentrated on the first kind, and field studies found that real-time arrival information reduces both perceived and measured waiting time and raises satisfaction [13], [14]. The second kind has received far less attention in deployed tools, even though crowding is one of the most frequently reported sources of discomfort in public transport and has a direct effect on safety, accessibility for elderly riders, and the willingness of riders to travel off-peak.

Operators face a mirror image of the same problem. Vehicles are often overfull during the morning and evening peaks and lightly used at other times, yet schedules and fleet assignments are rarely revisited because the evidence needed to justify a change is scattered across registers, spreadsheets, and staff experience. A shared digital record of vehicles, routes, positions, and crowd estimates would make such decisions easier to support.

Prior studies tend to address only one side of this picture. Tracking systems report vehicle positions [1], [4]; passenger flow studies demonstrate that demand varies systematically with time and route and can be learned from data [2], [5], [10]; and integrated designs have been proposed [3] without always being simple to reproduce. Smaller cities and institutions in particular need a solution that can be assembled from free, well-documented components and run on modest hardware.

The system presented here combines both forms of information in one browser-based tool. Positions are pushed to the client the moment the server records them, and every trip is annotated with an expected crowding category produced by a supervised model. The prototype runs on an academic route network in Kolhapur, India, uses simulated positions and synthetic demand, and is structured so that real feeds can replace the simulated ones without redesign.

The contributions of this paper are:

• An end-to-end design that unites live vehicle display, crowd classification, travel-time advice, and network administration under one authentication and authorisation scheme.

• A five-layer architecture in which the position source and the learning model are isolated behind narrow interfaces, so that either can be replaced independently.

• A comparison of three classifiers under a chronological split, with evaluation by macro F1-score and analysis of the error structure.
• An occupancy forecasting extension that uses live counts to predict crowding fifteen minutes ahead and is benchmarked against a no-change baseline.
• An evaluation that separates measured outcomes from design targets and states openly what the synthetic data can and cannot show.

Section II reviews related research. Section III defines the problem and goals, and Section IV summarises the requirements. Section V presents the design and the formal definition of the prediction task, Section VI describes the implementation, and Section VII reports the evaluation. Section VIII discusses the findings, Section IX lists limitations and planned work, and Section X concludes.

II. Literature Survey

Relevant work falls into five strands: vehicle tracking and rider information, prediction of passenger demand, systems that join tracking with prediction, communication techniques for live web applications, and automatic counting of passengers.

A. Vehicle Tracking and Rider Information

Monteiro, Silva, and Fernandes [1] outline a monitoring arrangement in which on-board receivers report coordinates to a server that republishes them on a web page; this source-server-viewer pattern is the template followed by most later systems, including the present one. Hannan et al. [4] extend monitoring to fleet management and argue that a single position stream can serve riders and supervisors at once. The OneBusAway project [13] showed at city scale that riders use arrival information when it is easy to reach, and the follow-up study of Watkins et al. [14] quantified the reduction in waiting time it produces. Operators now commonly publish positions in the GTFS Realtime format [15], which gives future versions of this work a standard input to consume.

B. Prediction of Passenger Demand

Reddy, Sharma, and Gupta [2] and Kumar and Singh [5] use supervised learning to predict passenger flow and find that hour of day, weekday, and route are the dominant explanatory variables. Fare-card transactions are the most common source of training data in this area; Bagchi and White [16] discussed their potential early on, and Pelletier, Trepanier, and Morency [17] surveyed their use for planning and modelling. Deep models have been applied where data are plentiful: Lv et al. [18] used stacked autoencoders for traffic flow, recurrent units such as the LSTM [19] capture longer temporal dependence, and DeepPF [10] targets metro ridership. Where features are few and tabular, decision-tree ensembles remain competitive and are cheaper to train and explain. Random forests [6] average many randomised trees, while gradient boosting [11] adds trees sequentially to correct remaining error; XGBoost [20] and LightGBM [12] made boosting fast on large data through regularisation and histogram binning.

C. Joint Tracking and Prediction

Kumar, Verma, and Singh [3] describe a platform that pairs vehicle tracking with a demand model. The argument for joining the two is practical: a position without a crowd estimate tells a rider when to walk to the stop but not whether boarding will be comfortable, while a crowd estimate without a position cannot be tied to a specific vehicle. The present design adopts this pairing and adds administration of the underlying network data.

D. Live Web Communication and Service Layering

A browser that must display moving objects can either ask the server repeatedly for updates or keep a channel open on which the server announces changes. The first option multiplies requests and delays each change by up to one polling period. The WebSocket standard [8] implements the second option over a single long-lived connection. Organising the server side as stateless resources exchanged through uniform interfaces, as described by Fielding [9], keeps the client, the business logic, and storage loosely coupled and individually testable.

E. Automatic Passenger Counting

Observed counts are needed both to train demand models and to report present conditions. Single-stage detectors that began with YOLO [21] can find people in video in real time on commodity processors, and trackers such as ByteTrack [22] link detections between frames so that a person crossing a door line is counted only once. Open implementations [23] lower the cost of experimentation, although results inside vehicles are sensitive to camera height, lighting changes, and occlusion in dense crowds.

F. Observations and Research Gap

Three observations follow from this review. First, rider information tools emphasise arrival times and seldom present crowding. Second, demand prediction studies report model accuracy but rarely deliver predictions to riders. Third, operators lack a lightweight shared record of vehicles, routes, and conditions. A deployable, low-cost system that addresses all three together is missing. Table I positions the proposed system against representative alternatives.

TABLE I. POSITIONING AGAINST REPRESENTATIVE APPROACHES

Approach	Vehicle position	Crowd estimate	Server push	Network admin	Principal weakness
Fixed paper timetable	No	No	No	No	Ignores delays and disruptions
Telephone or counter enquiry	No	No	No	No	Slow; depends on staff
Position-only tracking [1], [4]	Yes	No	Varies	Partial	Silent on crowding
Arrival-time applications [13], [15]	Yes	Rare	Yes	No	Need operator feeds
On-board counters [21], [22]	No	Present only	No	No	Hardware cost; no foresight
Demand models [2], [5], [10]	No	Yes	No	No	Not delivered to riders
Integrated platforms [3]	Yes	Yes	Varies	Partial	Hard to reproduce
This work	Yes	Yes	Yes	Yes	Simulated inputs at present

III. Problem Statement and Objectives

A. Problem Statement

Riders of city buses lack timely knowledge of vehicle positions and of expected crowding, and operators lack a single, current record of their vehicles and routes. The resulting uncertainty lengthens waiting, concentrates riders on already crowded trips, leaves other

trips under-used, and slows administrative response. The problem addressed here is to build an affordable system that reports bus positions continuously, estimates crowding for each trip in advance, recommends less crowded travel times, and gives authorised staff a controlled way to maintain network data.

The specific difficulties are:

- **Position uncertainty:** riders cannot see the vehicle and therefore cannot judge when to reach the stop.
- **Crowding uncertainty:** riders cannot tell in advance which trips will be full and which will have space.
- **Lack of guidance:** no advice is available on when to travel to avoid crowding.
- **Fragmented records:** information about vehicles, routes, and conditions is not held in one consistent store.

B. Objectives

The work set out to:

- Build a browser-based monitoring tool that needs no installation on the rider's device.
- Generate vehicle movement along real road geometry at a fixed cadence and show it on a map without reloading the page.
- Train and serve a supervised model that assigns each trip one of three crowding categories from time and route attributes.
- Turn stored predictions into a suggested departure time with lower expected crowding.
- Deliver position changes through a server-push channel rather than client polling.
- Protect administrative operations with hashed credentials, signed tokens, and server-side role checks.

IV. Requirement Analysis

Requirements were derived by walking through the tasks of a rider and of an administrator, then grouping the resulting needs into functional behaviour and quality attributes.

A. Stakeholders

Table II lists the parties who interact with the system and what each expects from it.

TABLE II. STAKEHOLDERS AND THEIR NEEDS

Stakeholder	Profile	Main needs
Rider	Member of the public on a phone or computer browser; no training assumed	Sign in, follow buses on a map, see expected crowding, find routes, get a suggested time
Administrator	Staff member who maintains network data	Create and edit vehicles and routes, watch the fleet, review predictions
Background services	Scheduled and event-driven server components	Advance simulated positions, persist them, broadcast them, request predictions

B. Functional Requirements

The system shall: (F1) allow riders to register and authenticate, and restrict administrative functions to the administrator role; (F2) display current vehicle positions on a map that updates without reloading; (F3) report a Low, Medium, or High crowding category for a chosen trip; (F4) present route details and propose a departure time with lower expected crowding; and (F5) let administrators create, modify, and inspect vehicles, routes, and stored predictions.

C. Quality Requirements

Ordinary requests should complete within about two to three seconds, and vehicle positions should refresh roughly every five seconds. Credentials must never be stored in readable form, and every protected operation must be authorised on the server rather than merely hidden in the interface. The interface must adapt to phone, tablet, and desktop screens. For maintainability, replacing the position source or the model must not require changes elsewhere. Section VII distinguishes which of these targets were measured and which remain design goals.

D. Constraints and Assumptions

The project was built with free software on student laptops. No GPS hardware, live operator feed, or real passenger counts were available, so positions are simulated and the model is trained on generated data. Riders are assumed to have a modern browser and network access, and the database, application server, and model service are assumed to be running whenever the application is in use.

V. Proposed System and Methodology

A. Overview of Operation

After signing in, a rider chooses a route or an origin and destination. The server collects the matching vehicles and their latest positions from the database, obtains a crowding estimate for each from the model service, and returns everything in one response. From then on, every position the server records is broadcast to the rider's browser, which moves the corresponding marker. A rider may also ask when to travel; the server then examines past predictions for that route and proposes the closest time slot in which crowding was low. Administrators use the same application to edit vehicles and routes and to monitor the fleet.

B. Architecture

Fig. 1 shows the five-layer architecture. Each layer communicates only with its neighbours through a defined interface, which confines the effect of a change to one layer and lets every layer be built and checked separately.

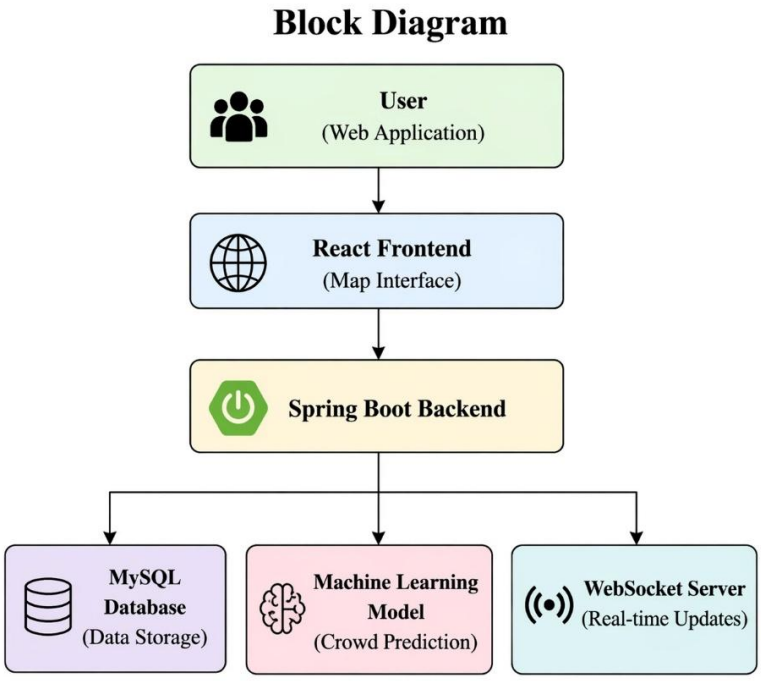


Fig. 1 Layered architecture of the proposed system

1) Client layer. A single-page React application renders the sign-in screens, the live map, crowding badges, route and travel pages, and the administrator console. It exchanges JSON with the server over HTTPS and holds one WebSocket subscription for position events. Layout adapts to the screen width so that the same code serves phones and desktops.

2) Application layer. A Spring Boot server exposes resource-oriented endpoints for identity, vehicles, routes, positions, crowding, and travel advice. It validates every input, verifies the caller's token and role, coordinates calls to storage and to the model service, and hosts the scheduler that advances simulated vehicles.

3) Learning layer. A small Python service wraps the trained classifier. It accepts trip attributes, checks that they lie inside the range seen during training, and returns a category together with the model version. Running it as a separate process keeps the Java and Python toolchains apart and allows retraining without redeploying the server.

4) Push layer. The server keeps a WebSocket endpoint to which clients subscribe after authenticating. Each newly recorded position is published once and delivered to all subscribers, so traffic grows with the number of position changes rather than with the number of refresh requests.

5) Storage layer. MySQL holds users, vehicles, routes and stops, position history, and every prediction issued. Foreign keys enforce consistency between vehicles, routes, and positions, and a composite index on vehicle and time keeps the lookup of the latest position fast as history grows.

C. Design Rationale

Three design choices deserve explanation. First, push delivery was chosen over polling because its cost is proportional to the number of changes. With N open clients polling every T seconds, the server handles N/T requests per second even when nothing has moved; with push, each recorded position produces one message per subscriber and no idle requests. For example, with a fleet of ten buses updated every five seconds and 200 riders polling every two seconds, the server would answer 100 requests per second, most of them returning unchanged data; with push it produces two position events per second and fans each out to the open connections, and a change reaches the map without waiting for the next poll.

Second, the model service is isolated behind HTTP so that the learning stack can evolve freely. The application server depends only on the request and response format, not on the algorithm, so the classifier can be changed, for example from a linear baseline to gradient boosting, without touching Java code. Third, every prediction is persisted. This produces an audit trail of what riders were told, supplies the history used for travel advice, and will allow predictions to be compared with observed counts once real counts are available.

D. Interaction Flow

Fig. 2 traces a rider's request. The client forwards the origin and destination, the server reads vehicles and positions from storage, sends trip attributes to the model service, and returns positions and crowding together. Independently of any request, the scheduler records each new position and pushes it to subscribed clients.

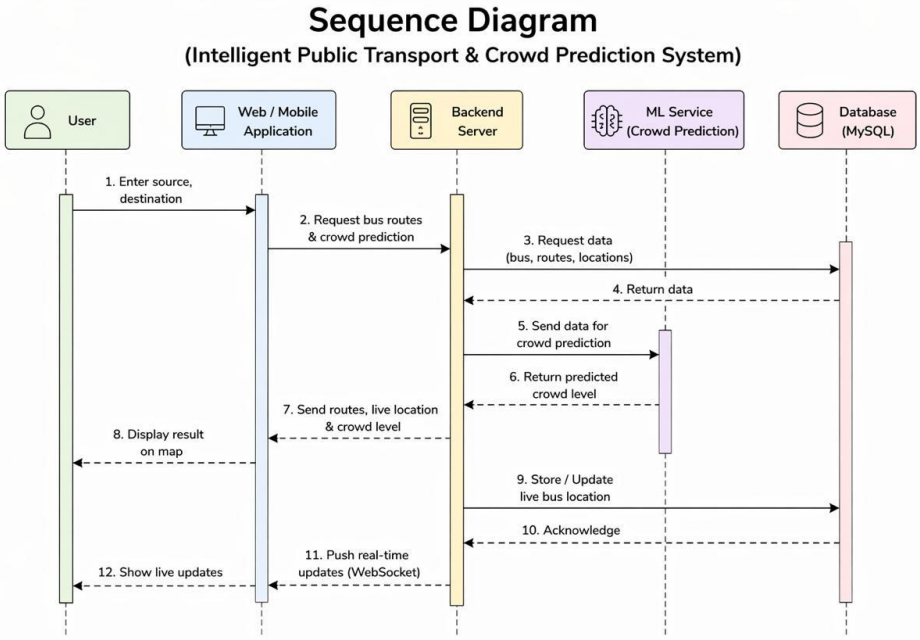


Fig. 2 Sequence of a rider request and of live position delivery

E. Functional Modules

Table III maps the functional requirements to the modules that implement them.

TABLE III. MODULES AND RESPONSIBILITIES

Module	Responsibility
Identity and access	Registration with validation, password hashing, sign-in, token issue, role checks on every protected call
Movement simulator	Advances each active vehicle along its road geometry every five seconds and records the new position
Live broadcast	Publishes each recorded position to subscribed clients over the WebSocket channel
Crowd estimation	Builds the feature vector, calls the model service, stores and returns the category
Travel advice	Groups stored predictions for a route by time slot and proposes the nearest low-crowding slot
Network administration	Creation, update, and review of vehicles, routes, stops, and prediction history by administrators

F. Formal Definition of the Prediction Task

Let P denote the number of passengers on board a trip and K the seating-plus-standing capacity of the vehicle. The occupancy ratio is

$$r = P / K \tag{1}$$

and the crowding category y is obtained by thresholding r:

$$y = \text{Low if } r < 0.50;\quad \text{Medium if } 0.50 <= r < 0.85;\quad \text{High if } r >= 0.85 \tag{2}$$

The thresholds reflect a practical reading of comfort: below half capacity most riders find a seat, between half and 85% standing becomes common, and above 85% boarding becomes difficult. Because hour of day is circular, it is represented by a pair of coordinates on the unit circle so that 23:00 and 00:00 are neighbours:

$$h_sin = sin(2\ pi\ h / 24),\quad h_cos = cos(2\ pi\ h / 24) \tag{3}$$

The feature vector for a trip is x = [h, h_sin, h_cos, d, w, L, S, K], where d is the weekday, w a weekend indicator, L the route length in kilometres, and S the number of stops. The learning problem is to find a function f such that f(x) approximates y. The passenger count and the ratio r are excluded from x because they define y and would make the task trivial.

Gradient boosting builds f as an additive sequence of M shallow regression trees fitted to the gradient of a multi-class logarithmic loss:

$$F_m(x) = F_(m-1)(x) + v * t_m(x),\quad m = 1, ..., M \tag{4}$$

where t_m is the tree added at stage m and v is the learning rate. The histogram variant [12] first bins each feature into a small number of intervals, which reduces the cost of searching for split points and handles missing values natively by learning a default branch for them.

G. Algorithms

Movement simulation and crowd estimation are summarised below. Simulation interpolates along the stored road polyline of each route, so markers follow streets rather than straight lines, and wraps to the first point when the end of the route is reached.

```
Algorithm 1: Movement simulation
every 5 s:
  for each active bus b:
    b.pos <- next point on polyline(route(b))
    if end of polyline: b.pos <- first point
    store (b.id, b.pos, now) in locations
    publish (b.id, b.pos, now) to subscribers
```

```
Algorithm 2: Crowd estimation
input: departure time t, route q, bus b
x <- features(t, q, b)          # equation (3) and list above
if any x_i outside training range: reject
y <- f(x)                       # Low / Medium / High
store (t, q, b, y, model_version)
return y
```

Travel advice estimates the category for every service hour of the day on the chosen route and ranks the hours in the requested window by expected crowding, best first. Alongside this ranking it shows the predictions stored for that route over the previous thirty days, grouped by hour, so that the rider can see the pattern behind the recommendation.

H. Technology Choices

Table IV lists the components used. All are free to use and widely taught, which lowers the barrier for other institutions that wish to reproduce the system.

TABLE IV. IMPLEMENTATION COMPONENTS

Concern	Component	Role in the system
User interface	React, HTML, CSS, JavaScript	Single-page client, map view, console
Mapping	Leaflet with OpenStreetMap; Google Maps optional	Tiles, markers, and route polylines
Application server	Java, Spring Boot	Endpoints, scheduling, security, push channel
Live delivery	WebSocket	Server-initiated position events
Persistence	MySQL	Relational store with keys and indexes
Learning	Python, scikit-learn, pandas, NumPy, Flask	Training, evaluation, inference endpoint
Counting (extension)	YOLO11n, ByteTrack, OpenCV	Door-line passenger counting from video
Security	Hashed passwords, JSON Web Tokens	Authentication and authorisation

VI. Implementation Details

A. Client Application

The client is organised into pages for sign-in, the rider dashboard, live tracking, crowding, travel information, and administration, with shared hooks for authenticated requests and for the live subscription. When a position event arrives, only the affected marker is moved, which keeps rendering cheap as the fleet grows. Crowding is conveyed by both colour and a text label so that it remains readable for colour-blind users. Fig. 3 shows the tracking page with the fleet list on the left and the selected route drawn along the road network.

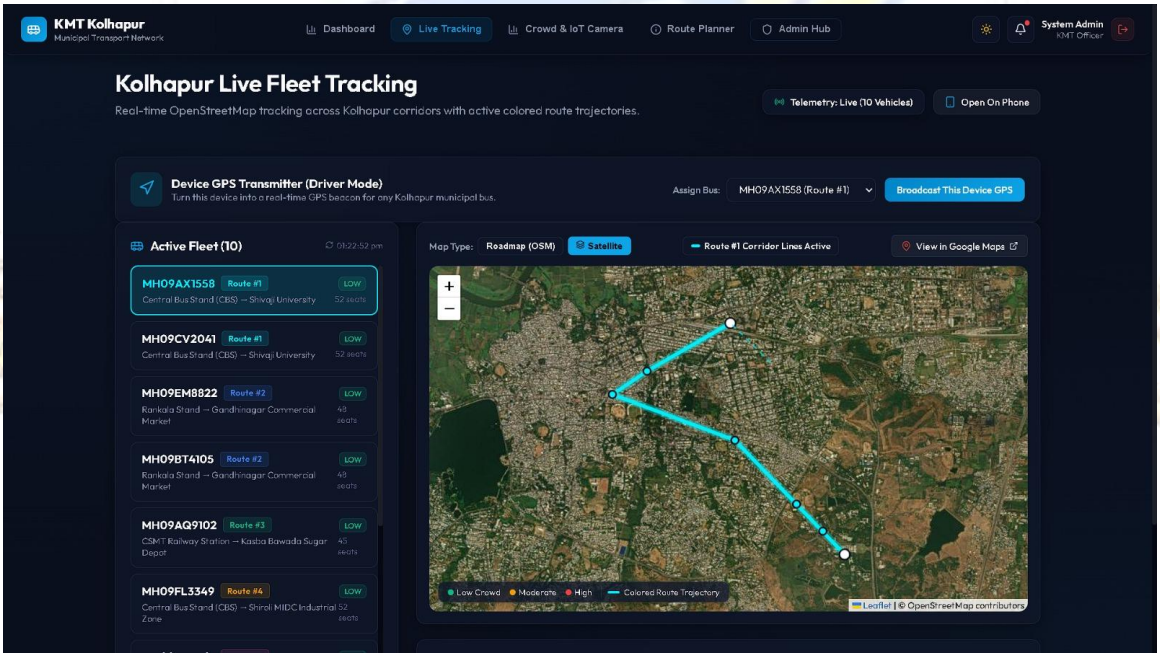


Fig. 3 Live tracking page showing the fleet list and a route on the demonstration network

B. Server and Interfaces

Endpoints are grouped by resource as shown in Table V. Read operations on vehicles and routes are open to any signed-in user, while all write operations and the administrative console require the administrator role. The same token that authorises HTTP calls is required to open the WebSocket connection, so an anonymous client cannot subscribe to positions. Inputs are validated before they reach the database, and failures are returned with explicit messages rather than generic errors.

TABLE V. ENDPOINT GROUPS

Base path	Operations	Who may call
/api/auth	Register, sign in, read own profile	Anyone / signed-in user
/api/buses	List, read, create, edit, remove vehicles	Read: user; write: admin

Base path	Operations	Who may call
/api/routes	List and read routes and stops; create and edit routes	Read: user; write: admin
/api/location	Latest positions, history, position upload from a device	User; upload: admin
/api/crowd	Predict, prediction history, route forecast, model details	Signed-in user
/api/travel	Search between stops, suggested departure time	Signed-in user
/api/dashboard, /api/admin	Statistics, alerts, users and roles, counting sensors	Administrator

C. Security Measures

Passwords are hashed with BCrypt, a deliberately slow adaptive function, before storage, so a leaked table does not reveal them. A successful sign-in returns a signed token carrying the user identity and role and an expiry time; the client attaches it to every request and the server verifies the signature and expiry before any processing. Authorisation is decided on the server for each endpoint, which closes the common gap in which a hidden button is mistaken for access control. Counting devices authenticate with their own per-sensor key that an administrator can rotate if a device is lost.

D. Training Data Generation

Since no ridership records were available, a generator produced 60,000 trips over 180 days from an explicit demand model. Expected load on a weekday combines two commuter peaks near 08:30 and 18:00 with a smaller midday bump; weekends use a flatter, lower curve. Load is scaled up for longer routes and routes with more stops, and smaller vehicles reach a given ratio sooner. Realism is added through multiplicative noise of about 12%, random surges on about 3% of trips to imitate events, and deletion of route attributes on about 2% of records to exercise missing-value handling. The resulting labels are imbalanced, with 39,745 Low, 10,423 Medium, and 9,832 High trips, which mirrors the fact that most real trips are not crowded. A fixed random seed makes generation and training repeatable.

E. Model Training Procedure

Records were ordered by date and divided so that the first 48,000 trips formed the training period and the last 12,000 the test period. A chronological split was preferred to a random one because random splitting places trips from the same day in both sets and inflates the score. Logistic regression, random forest, and histogram gradient boosting were each scored by five-fold cross-validation on the training period using macro F1:

$$F1_c = 2 * Prec_c * Rec_c / (Prec_c + Rec_c), \quad macro\ F1 = (1/3) * sum\ over\ c\ of\ F1_c \tag{5}$$

Macro averaging gives each category equal weight, so a model cannot score well simply by predicting Low for the majority. The best model was refitted on the full training period, evaluated once on the test period, and saved together with its version, feature list, and metrics so that the server can report which model produced each prediction.

F. Live Occupancy and Short-Term Forecast

An optional extension estimates how many people are on board now. A counting agent runs the YOLO11n person detector [21], [23] on a camera feed and uses ByteTrack [22] to follow each person, incrementing or decrementing the count when a track crosses a virtual line at the door. The agent reports counts to the server, which records them as observations and keeps a running occupancy per vehicle; the annotated video frame is relayed only to the administrator's live view and is never stored. Free seats are reported only when a seat sensor or a manual count supplies them; they are never guessed from detections. The counter has so far been exercised on a recorded clip and a laptop camera only, so no counting accuracy is claimed.

A second model uses the live occupancy to forecast both the passenger count and the category fifteen minutes ahead. A fourth category, Full, is added for ratios of 1.0 and above. Its inputs extend the time and route features with the present count, the present ratio, and the change over the previous fifteen minutes. It was trained on 202,560 synthetic occupancy records with a chronological split and compared with a persistence baseline that predicts no change. Error in passenger count is measured by mean absolute error and root mean square error:

$$MAE = (1/n) * sum\ |P_i - P^_i|, \quad RMSE = sqrt((1/n) * sum\ (P_i - P^_i)^2) \tag{6}$$

G. Demonstration Network and Storage

The prototype is loaded with eight illustrative routes linking well-known locations in Kolhapur, with road geometry taken from OpenStreetMap so that simulated buses move along streets. These routes are intended for demonstration and are not official routes or timetables. All entities live in related MySQL tables: users, buses, routes and route stops, locations, crowd predictions, model versions, and, for the extension, occupancy, sensors, and count observations. Old position records can be archived after a retention period to keep the table compact.

VII. Testing and Evaluation

Evaluation addressed three questions: does the application behave as specified, how well does the crowding classifier generalise to unseen days, and does live occupancy improve short-term forecasting over a naive rule.

A. Functional Verification

Table VI lists representative scenarios. Automated suites covering these scenarios were executed during integration: the server suite passed 35 of 35 tests, the browser end-to-end suite passed 18 of 18 with no console errors, and a dedicated check confirmed position delivery over the WebSocket channel. The model service suite was re-run while preparing this paper and passed 20 of 20 tests.

TABLE VI. REPRESENTATIVE VERIFICATION SCENARIOS

ID	Scenario	Expected behaviour	Verified by
V-01	New rider registers with valid data	Account created and confirmed	Server and browser suites
V-02	Registration reuses an existing email	Rejected with an explanatory message	Server suite
V-03	Sign-in with a wrong password	Access refused, error displayed	Server and browser suites
V-04	Rider calls an administrator endpoint	Server refuses the call	Server suite

ID	Scenario	Expected behaviour	Verified by
V-05	Rider opens the live map	Markers advance about every 5 s, no reload	Browser suite, push check
V-06	Rider requests a crowd estimate	Category returned and stored	Server and model suites
V-07	Estimate requested outside training range	Request rejected	Model suite
V-08	Administrator adds a bus and a route	Records saved and listed	Server and browser suites

B. Classifier Comparison

Table VII reports the cross-validated macro F1-score of the three candidates. Both tree ensembles far exceed logistic regression, whose linear decision boundaries cannot represent two separated daily peaks even with the cyclical encoding. Gradient boosting edged out the random forest and showed the smallest spread across folds, so it was selected.

TABLE VII. CROSS-VALIDATED MACRO F1-SCORE OF CANDIDATE MODELS

Model	Mean macro F1	Standard deviation
Logistic regression	0.594	0.016
Random forest [6]	0.881	0.004
Histogram gradient boosting (chosen)	0.885	0.003

C. Generalisation to Unseen Days

On the 12,000 trips of the test period, the chosen model scored 92.1% accuracy and 0.888 macro F1. The test score is close to the cross-validated score, which indicates that the model did not overfit the training period. Table VIII breaks the result down by category.

TABLE VIII. PER-CATEGORY RESULTS ON THE TEST PERIOD

Category	Precision	Recall	F1-score	Trips
Low	0.989	0.937	0.962	7,878
Medium	0.724	0.880	0.795	2,092
High	0.916	0.900	0.908	2,030

Low and High are identified with F1 above 0.90. Medium is the weakest category, chiefly through lower precision: some trips that were in fact Low were labelled Medium. This is expected, because Medium is a narrow band bounded by both thresholds, and noise in the generator moves trips across a threshold more easily than across two. Fig. 4 shows the full confusion matrix. Every error lies between adjacent categories; not a single Low trip was labelled High or the reverse. Of the 7,878 Low trips, 499 were labelled Medium, and of the 2,030 High trips, 202 were labelled Medium. For a rider, such mistakes are mild, since a High trip shown as Medium still signals a busy bus.

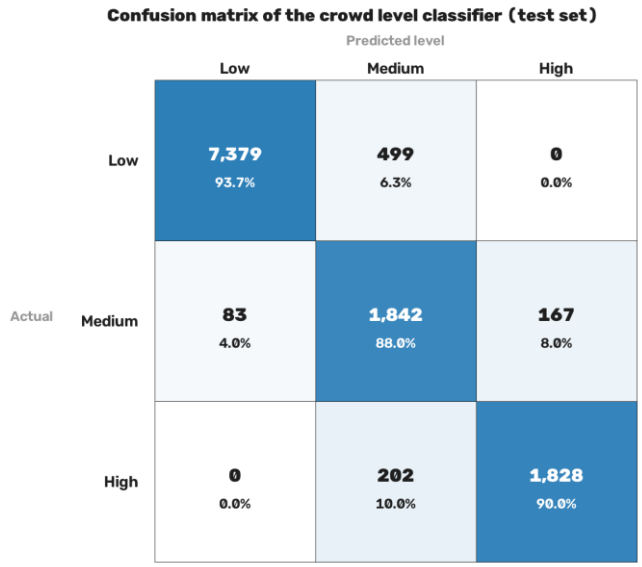


Fig. 4 Confusion matrix on the test period (counts and row percentages)

D. Short-Term Forecast

Table IX compares the fifteen-minute forecast with the persistence baseline on 50,640 test records. Using live occupancy together with time and route context cut the mean absolute error by about 28%, from 2.93 to 2.12 passengers, and lifted macro F1 from 0.728 to 0.789. The largest gain is in the High category, where F1 rose from 0.391 to 0.563. This is the transition riders most need to anticipate, because the baseline by construction cannot foresee a bus filling up. High nevertheless remains the hardest category, since it is a narrow band just below Full.

TABLE IX. FIFTEEN-MINUTE FORECAST AGAINST THE NO-CHANGE BASELINE

Metric	Baseline	Gradient boosting	Change
MAE (passengers)	2.93	2.12	-0.81
RMSE (passengers)	3.99	2.87	-1.12

Metric	Baseline	Gradient boosting	Change
Accuracy (category)	88.8%	89.6%	+0.8 pts
Macro F1 (category)	0.728	0.789	+0.061
F1, Low	0.955	0.956	+0.001
F1, Medium	0.764	0.787	+0.022
F1, High	0.391	0.563	+0.172
F1, Full	0.801	0.850	+0.049

E. Performance Targets

Table X lists the responsiveness targets and how each should be measured. They are design goals that have not yet been measured under controlled load; when they are, results should be reported with the hardware, the number of vehicles and users, and both the mean and a high percentile, because averages alone hide occasional long delays.

TABLE X. RESPONSIVENESS TARGETS

Quantity	Target	How to measure
Ordinary request	About 2-3 s	Time from request to response for sign-in, listing, and saving
Position cadence	About 5 s	Interval between successive stored positions of a vehicle
Push latency	Below 1 s	Time from storing a position to the marker moving
Crowd estimate	Interactive	Round trip including the call to the model service
Concurrent viewers	Several	Open many clients and record errors and resource use

F. Threats to Validity

The main threat is construct validity: the labels come from a generator whose assumptions the model can learn exactly, so high scores show that the pipeline works, not that real crowding is this predictable. Real demand also depends on weather, festivals, school calendars, and service disruptions that the generator does not model. External validity is limited by the single demonstration network. Internal validity is supported by the chronological split, the fixed seed, and the use of macro F1, which prevents the majority category from masking poor minority performance.

VIII. Discussion and Applications

A. Interpretation of Results

Two findings carry over beyond the synthetic setting. First, the gap between linear and tree-based models shows that crowding is a strongly non-linear function of time and that any practical model must be able to represent separate peaks; this is consistent with the patterns reported in passenger flow studies [2], [5]. Second, the structure of the errors matters as much as their number. A model whose mistakes stay between adjacent categories is safe to show to riders even when its accuracy is imperfect, whereas a model that occasionally calls a full bus empty would quickly lose their trust. Reporting the confusion matrix alongside accuracy is therefore recommended for any rider-facing crowding service.

The forecasting experiment suggests a staged deployment path. With only timetables and route data, the time-and-route classifier gives a reasonable prior. Once even a few vehicles carry counters, live occupancy can be used to refine short-term expectations, and the improvement over persistence indicates that this refinement is worthwhile precisely when buses are filling up.

B. Comparison with Related Systems

Compared with position-only trackers [1], [4], the system adds a forward-looking estimate of crowding. Compared with demand models reported in isolation [2], [10], it delivers the estimate to riders at the moment of decision and records every estimate for later audit. Compared with integrated designs [3], it is built entirely from free components, documents its data assumptions, and separates measured results from targets, which makes it easier for other institutions to reproduce and extend.

C. Privacy and Ethical Considerations

A deployment that uses cameras must respect rider privacy. Counting should take place on the device, only aggregate counts should leave the vehicle, and riders should be informed through notices on board. Position data concerns vehicles rather than individuals, but account data must be protected and retained only as long as needed. Crowding estimates should be presented as guidance, with the uncertainty made clear, so that riders do not rely on them for decisions with safety implications.

D. Applications

- **Municipal bus services:** riders check position and expected crowding before leaving home and can choose a less crowded trip.
- **Campus and corporate shuttles:** colleges and industrial estates can run the same platform for their own fleets.
- **Planning support:** stored predictions, and later observed counts, highlight routes and hours that need more capacity.
- **Accessibility:** elderly riders and riders with disabilities can avoid trips likely to be crowded.
- **Teaching and research:** the modular code base is a practical testbed for transport data science courses.

IX. Limitations and Future Work

A. Limitations

The prototype has not been connected to vehicle receivers or an operator feed; positions are simulated. Both models are trained on generated data and cannot yet be relied on to describe real conditions. The demonstration routes are illustrative. The counting extension has not been validated inside a moving vehicle. Responsiveness targets have not been measured under load. Ticketing, payment, crew scheduling, and maintenance are outside the scope of the work.

B. Future Work

Planned work includes: connecting real receivers through the existing position upload endpoint; consuming GTFS Realtime feeds [15]; retraining on fare-card records [16], [17] or on-board counts and adding weather, holiday, and event features; validating door-line counting in buses and reporting its error; estimating arrival times at stops; a mobile client with alerts when a chosen bus approaches; planner-facing reports; and load testing with horizontal scaling of the application server and the push channel.

X. CONCLUSION

This paper described a browser-based system that tells bus riders both where their vehicle is and how crowded it is likely to be, and gives administrators a controlled way to maintain network data. The five-layer design keeps the position source and the learning model behind narrow interfaces, so that the simulated inputs used today can be exchanged for real receivers and real counts. On a chronological split of synthetic trips, a histogram gradient boosting classifier achieved 92.1% accuracy and 0.888 macro F1, and all of its errors fell between neighbouring categories; a fifteen-minute occupancy forecast reduced passenger-count error by about 28% relative to a no-change rule. These results establish a working and reproducible pipeline. Field data will be needed to establish real-world accuracy, and the architecture has been prepared for that step.

XI. REFERENCES

[1] J. Monteiro, P. Silva, and R. Fernandes, "Real-time public transport monitoring system using GPS and web technologies," International Journal of Computer Applications, 2018.
[2] K. Reddy, V. Sharma, and A. Gupta, "Passenger flow prediction in public transport using machine learning techniques," in Proc. IEEE Conference on Intelligent Transportation Systems, 2019.
[3] S. Kumar, R. Verma, and P. Singh, "Smart public transport system with integrated tracking and prediction," International Journal of Smart Transportation, 2021.
[4] M. A. Hannan et al., "Intelligent bus monitoring and management system," IEEE Transactions on Intelligent Transportation Systems, 2018.
[5] S. Kumar and R. Singh, "Passenger flow prediction using machine learning," International Journal of Transportation Science and Technology, 2021.
[6] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, 2001, doi: 10.1023/A:1010933404324.
[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
[8] I. Fette and A. Melnikov, "The WebSocket protocol," RFC 6455, Internet Engineering Task Force, Dec. 2011.
[9] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000.
[10] Y. Liu, Z. Liu, and R. Jia, "DeepPF: A deep learning based architecture for metro passenger flow prediction," Transportation Research Part C: Emerging Technologies, vol. 101, pp. 18-34, 2019.
[11] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," The Annals of Statistics, vol. 29, no. 5, pp. 1189-1232, 2001.
[12] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in Advances in Neural Information Processing Systems 30, 2017, pp. 3146-3154.
[13] B. Ferris, K. Watkins, and A. Borning, "OneBusAway: Results from providing real-time arrival information for public transit," in Proc. SIGCHI Conference on Human Factors in Computing Systems (CHI), 2010, pp. 1807-1816.
[14] K. E. Watkins, B. Ferris, A. Borning, G. S. Rutherford, and D. Layton, "Where is my bus? Impact of mobile real-time information on the perceived and actual wait time of transit riders," Transportation Research Part A: Policy and Practice, vol. 45, no. 8, pp. 839-848, 2011.
[15] Google, "GTFS Realtime reference," General Transit Feed Specification. [Online]. Available: https://gtfs.org/realtime/
[16] M. Bagchi and P. R. White, "The potential of public transport smart card data," Transport Policy, vol. 12, no. 5, pp. 464-474, 2005.
[17] M.-P. Pelletier, M. Trepanier, and C. Morency, "Smart card data use in public transit: A literature review," Transportation Research Part C: Emerging Technologies, vol. 19, no. 4, pp. 557-568, 2011.
[18] Y. Lv, Y. Duan, W. Kang, Z. Li, and F.-Y. Wang, "Traffic flow prediction with big data: A deep learning approach," IEEE Transactions on Intelligent Transportation Systems, vol. 16, no. 2, pp. 865-873, 2015.
[19] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.
[20] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785-794.
[21] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779-788.
[22] Y. Zhang et al., "ByteTrack: Multi-object tracking by associating every detection box," in Proc. European Conference on Computer Vision (ECCV), 2022.
[23] G. Jocher, J. Qiu, and A. Chaurasia, "Ultralytics YOLO," software, version 8.3 (YOLO11), 2024. [Online]. Available: https://github.com/ultralytics/ultralytics